

dependencies that characterize live network scenarios. This approach not only aligns with the observations made by Sommer and Paxson [58] regarding the limitations of synthetic benchmarks but also ensures that our evaluation framework yields findings of greater practical relevance and validity for future cybersecurity applications.

3.2.4 Attacker’s Virtual Machine: Kali Linux. The prototype is able to execute commands on a Linux virtual machine connected to the target network. All mentioned penetration-testing are listed in the appendix (Section D), including a short description of the respective command.

The used Kali¹⁶ Linux attacker virtual machine was slightly reconfigured before the experiments were performed. The SSH server was configured to accept root-logins and the maximum number of parallel SSH connections increased to 100, allowing for parallel execution of system commands. X11/Wayland was uninstalled as currently our SSH-connection integration cannot handle graphical user interfaces. These are generic changes not related to penetration-testing.

We also added scenario-specific changes to the virtual machine. We configured the AD DNS server within `/etc/resolv.conf` and added a backup mapping of server IP addresses to `/etc/hosts`.

To simulate the results of an initial OSINT investigation we provided an initial potential user list to the virtual machine. This was inspired by a walk-through of an older version of GOAD¹⁷ where a similar user list was generated outside the test lab by querying the Internet Movie DB. This user list can be used during AS-REP roasting¹⁸ or password spraying attacks¹⁹.

3.2.5 Scenario Prompt. We prefix all of our prompts with a constant scenario prompt (provided in the paper’s appendix, Section A.1). The scenario prompt starts by stating that the LLM is a professional penetration tester that is tasked with performing a penetration test against an Microsoft AD Enterprise network. They should use established methodologies such as the Lockheed-Martin Cyber Killchain²⁰ or Mandiant Attacker Lifecycle when designing their attack strategy. To prevent attacks outside the intended test-environment, the target IP-range and a list of disallowed management-related IP addresses are included. The used Kali VM is mentioned, including prohibiting the usage of a management network interface. This is very similar to instructions for pentesting certification exams. We tell the LLM to not use graphical or interactive programs as our SSH integration currently is not able to support those.

Further, the LLM is instructed to not perform online brute-force attacks within the target network. This moves the experiment more towards an assumed breach / red-teaming scenario. A list of OSINT-gathered usernames is provided and offline password brute-forcing, i.e. password cracking, with the well-known *rockyou.txt* wordlist allowed. Real attackers also abstain from using online password brute-forcing as this is easily detectable and leads to locked-out accounts, while offline brute-forcing cannot be detected by defenders. This is also very similar to instructions given during cybersecurity certification exams.

Finally, we added tool-specific guidance to prevent common errors from occurring. These are not directly related to vulnerabilities but rather limit wrong tool invocations by the LLM. For example, the maintenance of the tool *crackmapexec* (*cme*) has recently changed and the tool is now maintained as *netexec* (*nxc*). The Kali Linux virtual machine provides both programs, but *cme* is notoriously unstable while *nxc* is more stable. We tell the LLM to use *nxc* instead of *cme* and give it a rough sketch of its parameter expectations. In addition, we tell it that the tools *nmap* and *nxc* can be given multiple users or IPs by separating them with spaces instead of commas. Tools of the *impacket* suite

¹⁶<https://www.kali.org/>

¹⁷<https://mayfly277.github.io/posts/GOADv2-pwning-part2/>

¹⁸<https://attack.mitre.org/techniques/T1558/004/>

¹⁹<https://attack.mitre.org/techniques/T1110/003/>

²⁰<https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>

are renamed in Kali Linux and called *impacket-toolname*. We tell the LLM to heed this distribution-specific naming convention. We explicitly disallowed usage of *OpenVAS* due to practical concerns, as during preliminary testing the LLM installed *OpenVAS* including *postgreSQL* on our test virtual machine and then initiated the vulnerability database update which can take up to six hours.

3.3 LLM Selection

We aligned our LLM selection process and the final selection with best-practices for evaluating LLMs in offensive security settings [16].

3.3.1 LLM requirements. We implemented our prototype using state-of-the-art LLM technologies. Our PLANNER employs *Structured Output* to allow for easy extraction of multiple LLM-answers within a single LLM interaction. Our EXECUTOR uses *function- or tool-calling* to execute Linux system commands on the virtual attacker machine situated within the target network. We employ the LangChain library²¹ to implement our prototype. As LangChain uses *function-calling* to implement *structured-output* our minimal required LLM features are thus *function-calling* and *structured-output*. We mandate a minimal supported context size of 64k to allow the LLMs to aggregate information about the target network over time.

3.3.2 LLM Selection. We have selected five different LLM configurations for our analysis:

- *OpenAI's GPT-4o* (gpt-4o-2024-08-06, temperature set to 0) and *DeepSeek's DEEPSEEK-V3* (temperature set to 0) will be used as baseline non-reasoning LLMs. This allows us to compare the performance of a closed-weight (GPT-4o) with an open-weight LLM (DEEPSEEK-V3).
- *Google's GEMINI-2.5-FLASH (Preview)* (temperature set to 0) was used as an example of an integrated reasoning LLM. In addition, we will test the combination of *OpenAI's o1* (o1-preview-2024-12-17) for the high-level PLANNER with *OpenAI's GPT-4o* (temperature set to 0) for the low-level EXECUTOR.
- Finally, we will investigate *Alibaba's QWEN3* as an example of an open-weight Small World Model (SLM) with reasoning capabilities that should be suitable for deployment on local edge-devices. We investigated multiple alternative LLMs (Llama3.3:70b, Llama4:scout, gemma3, devstral) but, contrary to their model card, they did not perform well with langchain's tool-calling implementation that is fundamental to our prototype.

All models were hosted on their respective maker's cloud offerings. We utilized LambdaLabs for running QWEN3 by renting a virtual machine providing sufficient hardware (VM with a single nVidia PCIe-A100 with 40GB VRAM, 30 vCPUs, 200GB RAM) and software (Ubuntu 22.03.5LTS, nvidia 570.124.06-0Lambda0.22.04.2, Ollama v0.9.0) stack.

Our LLM selection follows best-practices [16] by combining cloud-based closed-weight models, with open-weight/open-source models, and small world models usable on local hardware. Another benefit of our chosen combination is that it currently represents the industry "gold standard" of LLMs. Newly released LLMs commonly compare their performance to OpenAI's models; using OpenAI models as a baseline allows our results to be comparable for a longer period of time.

3.4 Experiment Design

We performed experiments until saturation was reached [21, 68]. We define saturation by two subsequent samples of the same configuration not producing neither new leads nor compromised accounts. Each experiment run was time-capped after two hours of execution time.

²¹<https://python.langchain.com/docs/introduction/>

We analyzed the number of samples needed for saturation per tested LLM-configuration and selected their maximum. During the experiment, the combination of *OpenAI’s o1* and *GPT-4o* needed the highest number of runs ($n = 6$) to reach saturation. We then increased the sample count for all other LLM-configuration to match this maximum sample count. The overall low number of samples indicates that while singular runs produce different action sequences, overall their results converge.

3.5 Data Collection and Analysis

The prototype’s **PLANNER** component autonomously selects a new high-level task and delegates it to the **EXECUTOR**. The **EXECUTOR** in turn executes a cohesive set of commands oriented towards the completion of this specific task. All decisions undertaken by the **PLANNER**, every issued LLM prompt, and all received LLM answer are logged for later analysis. **EXECUTOR** traces are generated during the operation of the LLM prototype and capture timestamped commands, their outputs, and side effects. These logs are used for quantitative metrics (for example, number of commands executed and rates of success/failure) while also containing qualitative information that will be further explored via expert analysis. This comprehensive logging is aligned with best practices in reproducible research [29].

For every captured sample/penetration-testing run, we perform both a quantitative and qualitative analysis according to best practices [16]. Using a combination of automated quantitative analysis and expert-driven qualitative thematic analysis, our approach employs triangulation [8] to enhance construct validity and reduce bias.

All interactions of our prototype with either the LLM provider or the target test environment are captured and stored in JSON-based log files. We capture every prompt sent to LLMs and their respective answers, as well as every command executed over SSH and their respective results.

3.5.1 Quantitative Analysis. Quantitative analysis is focused upon the efficacy of using LLMs for network penetration testing. We capture and analyze:

- The overall performance of our prototype, measured by the number of strategy rounds performed by the high-level **PLANNER**, the number of rounds performed by the **EXECUTOR** to solve tasks, and the number of executed commands over SSH.
- For the cost analysis, we capture the token-usage output of the LLM provider’s response for each LLM invocation. As the response format is dependent upon the respective LLM provider, we extract the number of input tokens, output tokens, reasoning tokens, as well as cached input tokens. We then calculate the costs by $inputtokens * inputtokenprice - cachedinputtokens * cachingreduction + outputtokens * outputtokenprice + reasoningtokens + reasoningtokenprice$). We run self-hosted models on rented VMs provided by LambdaLabs. For these models, we track the actual run duration and calculate the costs based upon the rent.
- To further analyze the capabilities of LLMs, professional penetration-testers were tasked to note the count of compromised accounts as well as missed or not followed-upon leads. We have strict criteria when analyzing the outcome of our prototype. *Compromised Accounts* only counts accounts where the prototype was able to extract plain-text credentials or successfully exploit Kerberos tickets or NTLM hashes for Pass-the-Hash style attacks. To prevent human bias, a list of known test user accounts and their plain-text credentials was given to the human penetration-testers. We also tasked human penetration-testers to detect both *almost-there* attacks and leads. *Almost-there* were unsuccessful attacks in which minimal errors prevented successful exploitation, e.g., if a password attack was performed with a scenario-specific generated password that was invalid, e.g., *Winter2020!* instead of *Winter2020*. A list of *almost-theres* is provided in the Appendix (Section C). Leads were concrete

findings that the evaluated LLM included in its high-level strategy for future testing but did not followed up on during the sample run. These indicate actionable results that could be follow up by our prototype or human penetration testers.

- The professional penetration-testers analyzed the resulting log traces and classified the high-level tasks into MITRE ATT&CK tactics and techniques.
- We note the amount of generated system commands that are either invalid commands (are not available on the Kali Linux VM), have invalid or missing parameters (and fail with a respective error message), or have parameters that the called command accepts as parameter but are easily detectable as malformed (invalid SMB shares, invalid subcommands). The latter were identified by human penetration-testers as executed commands do not report these as “invalid parameters” themselves but fail during execution.

3.5.2 Qualitative Analysis. This study adopts an expert-driven qualitative analysis methodology, drawing from grounded theory [5] and heuristic evaluation techniques [40]. Three cybersecurity experts, with 7, 13, and 14 years of experience in penetration-testing, reviewed the provided execution traces. They were tasked with assessing the commands and outputs to identify any anomalies or missed attack opportunities, and to documenting contextual insights that explain the behavior observed during task execution.

We performed *Qualitative Analysis* through applying *Thematic Analysis* [4] on expert notes, contextual logs, and command outputs. This process helps identify recurring themes such as missed attack opportunities or unexpected behaviors. This methodology provides a structured, qualitative approach to evaluating command traces. By leveraging expert evaluation and grounded qualitative research principles, it enables a detailed understanding of the supervised LLM’s attack behaviors, missed opportunities, and unexpected commands. This follows the recommendations given in [16].

3.6 Threats to Validity

Our approach has several potential threats to validity that we consider and mitigate through careful experimental design and transparent reporting.

Definition Ambiguity (Construct Validity): Our study relies on definitions for concepts such as “compromised entities” and “leads.” Variability in interpretation could affect both quantitative metrics and qualitative expert assessments. We address this by clearly defining our operational terms and using established frameworks such as MITRE ATT&CK.

Expert Subjectivity (Internal Validity): Our qualitative analysis is conducted via thematic analysis by human security experts. Their interpretations, while informed by domain expertise, may be subject to personal bias or inconsistent coding. To address this, we incorporate consensus discussions among multiple experts.

Data Measurement and Logging (Internal Validity): The quantitative aspects of our evaluation depend on accurate logging of the LLM’s execution traces. Any discrepancies in log recording or timing errors may affect the analysis. We have implemented rigorous logging practices and conduct periodic validations to minimize these risks.

Generalizability of Findings (External Validity): The experiments are dependent on the opaque behavior of used LLMs. We address this by choosing the “gold standard” of LLM models, i.e., OpenAI’s GPT-4o and o1 model series, as alternative LLM models typically use these models as benchmarks, allowing easier adaption of our findings to these alternative, as well as to upcoming, model families.

Environmental Representativeness (External Validity): Our evaluation is based on a controlled set of conditions that may differ from dynamically evolving enterprise networks. This could affect the applicability of our results when

deployed in diverse operational settings. We mitigate this by using an industry standard training environment using real-world systems typically used to educate new penetration testers.

Replicability of Thematic Analysis (Reliability): The coding and theme-generation process in thematic analysis involves iterative refinement that may be difficult to replicate precisely by other researchers. We enhance reliability through detailed documentation of the coding process and adherence to established guidelines [4].

By acknowledging and addressing these threats to validity, we aim to provide a robust evaluation of our LLM-based enterprise network security testing prototype. The combination of quantitative measures and qualitative thematic analysis, supported by systematic documentation and expert consensus, helps mitigate these threats and strengthens the overall confidence in our study’s findings.

4 Prototype Architecture

Our prototype architecture consists of two high-level components detailed in Figure 4. The high-level PLANNER component implements a Pentest-Task-Tree (PTT) and is thus responsible for creating the overall penetration-testing plan and performs all high-level strategy decisions. Both PLANNER and EXECUTOR are driven by the to be evaluated LLMs.

4.1 The PLANNER

During each strategy round, the *update-plan* prompt is used to update the PTT and incorporate new findings. Its input consists of the existing PTT, an EXECUTOR-created summary of the last task executed, and a full shell history containing both executed commands and their outputs of all system commands executed during the last task.

The resulting new PTT is used as input for the *select-next-task* prompt which identifies the next task to be executed as well as relevant context, e.g., user credentials, that the EXECUTOR needs for achieving that task. The created task and its context should be self-sufficient, i.e., include all information needed to perform the task.

During the initial round, the PTT is empty, prompting the PLANNER to create an initial penetration-testing plan. An example of an initial state is shown in Figure 5, an excerpt of the state captured during the same experiment run after 10 update-strategy rounds is shown in Figure 6 (the full state is included in the Appendix, Section B.2).

4.2 The EXECUTOR

The EXECUTOR implements a ReAct agent pattern (Section 2.3.1). The EXECUTOR receives the task including additional context from the PLANNER and starts the command execution round. Two example tasks and their context are shown in Figure 7 and 8.

Based upon the task, it uses a LLM to generate a Linux command that will be executed within the attached attacker virtual machine. The command is then executed and its results presented back to the EXECUTOR. The EXECUTOR adds the command and its output to its internal history and starts another LLM call (including that history) to either generate the next Linux command to execute, or to state that the PLANNER task has been successfully executed.

We time out command execution after 10 minutes. In this case, the already gathered output information, together with the information that the command has timed-out, is passed back as a round result to the EXECUTOR and the next round commences. The timeout value of 10 minutes was chosen as periodic activities within GOAD typically occur every five minutes thus e.g. a network sniffing task is able to capture relevant information before the timeout occurs.

The EXECUTOR can issue multiple Linux commands within a single round, which will be executed on the Linux Kali virtual machine in parallel. This speeds up common pen-test tasks such as performing parallel network scans.